BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC FPT

KHOA CÔNG NGHỆ THÔNG TIN — BỘ MÔN KỸ THUẬT PHẦN MỀM

OMNICAST | NỀN TẢNG PHÁT SÓNG THÔNG MINH

BÁO CÁO NGHIỆM THU ĐỒ ÁN HỌC PHẦN

HỌC PHẦN: PRN232 — XÂY DỰNG ỨNG DỤNG PHÍA SAU ĐA NỀN TẢNG

HỆ THỐNG LẬP LỊCH VÀ PHÁT SÓNG NỘI DUNG SỐ ĐA NỀN TẢNG OMNICAST

Enterprise Media & Broadcast Intelligence Network

Kiến trúc NestJS vi dịch vụ gRPC, Cổng Web Next.js 15, Thuật toán chống xung đột lịch EPG 24/7 và Trợ lý AI Curator

Giảng viên hướng dẫn:	Bộ môn Kỹ thuật Phần mềm (SE)
Học kỳ thực hiện:	Fall 2026 (FA26)
Nhóm tác giả:	OmniCast Engineering Group
Kho mã nguồn (GitHub):	https://github.com/TomOutfit/OmniCast_PRN232-PRM393_Project_Fa26Semester
Hệ thống trực tuyến:	https://omnicast-fe.vercel.app (API: /api/v1)
Quy mô & Kiểm định:	**142.897 dòng mã (LOC) | 482/482 Tests PASS (100%)**

HÀ NỘI / TP. HỒ CHÍ MINH — THÁNG 10 NĂM 2026

Xây dựng ứng dụng phía sau đa nền tảng: Hệ thống lịch phát và phát sóng nội dung số OmniCast

Nhóm Kỹ Thuật OmniCast (OmniCast Team) — PRN232 (Fa26)

Bộ môn Kỹ thuật Phần mềm, Trường Đại học FPT
Kho mã nguồn: https://github.com/TomOutfit/OmniCast_PRN232-PRM393_Project_Fa26Semester
Hệ thống trực tuyến: https://omnicast-fe.vercel.app

***Abstract*— OmniCast là một hệ thống lịch phát và phát sóng nội dung số đa nền tảng, xây dựng cho môn PRN232. Hệ thống gồm một back-end NestJS với 19 module nghiệp vụ và 140 endpoint REST, một cổng web Next.js 15 với 33 route, và một microservice ghi nhật ký kiểm toán giao tiếp qua gRPC. Ba đóng góp chính được trình bày: thuật toán phát hiện xung đột lịch phát thực hiện bằng một truy vấn SQL thô tính thời điểm kết thúc hiệu lực của sự kiện; cơ chế điền lịch EPG 24/7 với lựa chọn ngẫu nhiên có hạt giống bảo đảm mỗi lưới kênh-ngày luôn đầy đủ và có tính tái lập; và quy trình biên tập AI gọi mô hình ngôn ngữ lớn một lần duy nhất, kiểm tra đầu ra theo schema Zod, có đường dự phòng heuristic và dấu vết phiên bản đầy đủ. Lược đồ cơ sở dữ liệu gồm 21 model Prisma trên PostgreSQL. Toàn bộ hệ thống đã được triển khai trên Vercel và kiểm thử bằng 482 kiểm thử tự động trong quy trình tích hợp liên tục năm công việc; tất cả đều đạt.**

***Index Terms*—broadcast scheduling, EPG, NestJS, Prisma, Next.js, gRPC, RBAC, Flutter, AI curation, conflict detection, offline-first, CI/CD.**

I. Mở đầu

Trong những năm gần đây, nhu cầu theo dõi nội dung truyền hình trực tuyến tại Việt Nam đã thay đổi rõ rệt. Người dùng không còn chỉ ngồi trước tivi lúc 21 giờ; họ xem các chương trình theo dõi ngẫu nhiên, bỏ lỡ lịch phát của kênh mình yêu thích, và muốn được gợi ý nội dung phù hợp với sở thích. Ở phía bên kia, các đài truyền hình và nền tảng nội dung số cần một hệ thống lịch phát thống nhất, đáng tin cậy và có thể mở rộng để phục vụ đồng thời nhiều nền tảng: web, ứng dụng di động và các thiết bị phát trực tuyến.

Bài báo này trình bày **OmniCast** — một hệ thống lập lịch và phát sóng nội dung số đa nền tảng, được xây dựng cho môn **PRN232 — Building Cross-Platform Back-End Application**. Hệ thống gồm một back-end NestJS hiệu năng cao phục vụ đồng thời một cổng web Next.js 15 và một ứng dụng Flutter, với lõi nghiệp vụ tập trung vào ba năng lực chủ chốt: lịch phát điện tử (EPG) không bao giờ để trống, cơ chế phát hiện xung đột lịch phát, và trợ lý biên tập AI có kiểm duyệt theo chuẩn đạo đức nội dung.

A. Động cơ và vấn đề cần giải quyết

Ba vấn đề kỹ thuật chi phối toàn bộ thiết kế của hệ thống được xác định từ yêu cầu thực tế của dự án.

1) **Lịch phát thưa thớt.** Một kênh truyền hình thực tế chỉ có khoảng một chương trình gốc mỗi ngày. Nếu dựng lưới EPG 24 giờ trực tiếp từ bảng dữ liệu, phần lớn lưới sẽ trống và bộ lọc theo ngày ở giao diện người dùng trở nên vô nghĩa. Hệ thống cần một cơ chế điền lịch tự động, luôn đảm bảo mỗi kênh trong mỗi ngày đều đầy đủ 24 giờ nội dung hợp lệ.
2) **Xung đột lịch phát.** Nếu hai chương trình cùng một kênh có khoảng thời gian chồng lấn, người xem sẽ không biết chương trình nào được phát. Cơ chế kiểm tra phải phát hiện được cả ba dạng giao thoa (đầu, đuôi, bao trùm toàn bộ) và phải chạy trong một câu truy vấn cơ sở dữ liệu để tránh điều kiện tranh chấp.
3) **Kiểm duyệt nội dung không tự động.** Quy trình biên tập nội dung (mô tả, thẻ, phân loại) là thủ công và tốn thời gian. Việc tự động hoá bằng mô hình ngôn ngữ lớn giúp giảm tải, nhưng đòi hỏi cơ chế dự phòng khi không có khóa API, và đầu ra phải được kiểm tra theo schema trước khi lưu.

B. Đóng góp của bài báo

Bài báo đóng góp như sau:

1) Kiến trúc phân tầng **NestJS + Prisma + PostgreSQL** với 19 module nghiệp vụ, 140 endpoint REST và tài liệu OpenAPI tự sinh, kèm lớp bảo vệ RBAC bốn vai trò.
2) Thuật toán phát hiện xung đột lịch phát bằng một truy vấn SQL thô, tính toán thời điểm kết thúc hiệu lực bằng `COALESCE(endedAt, scheduledAt + duration)`.
3) Cơ chế điền lịch EPG 24/7 với lựa chọn ngẫu nhiên có hạt giống (seeded PRNG) theo cặp (ngày, kênh), bảo đảm cùng một ngày luôn cho cùng một lịch.
4) Quy trình biên tập AI gọi mô hình một lần duy nhất (`gpt-4o`), kiểm tra schema bằng Zod, có cơ chế thử lại và đường dự phòng heuristic.
5) Một microservice ghi nhật ký kiểm toán giao tiếp qua gRPC, tách biệt khỏi tiến trình API chính.
6) Cổng web Next.js 15 với 33 route và quản trị AI Curator dựa trên kiến trúc Next.js App Router.
7) Quy trình CI/CD năm công việc trên GitHub Actions với cổng chất lượng tự động, cùng 482 kiểm thử tự động trên ba bộ kiểm thử.

C. Phạm vi bài báo

Bài báo tập trung vào phần back-end và cổng web của hệ thống, là hai sản phẩm của môn PRN232. Phần ứng dụng di động Flutter được trình bày đầy đủ trong bài báo song song ứng với môn PRM393. Các khía cạnh triển khai hạ tầng, cấu hình biến môi trường và quy trình phát hành ứng dụng di động được trình bày trong tài liệu triển khai đi kèm, còn ở đây chỉ tóm tắt ở mức kiến trúc.

II. Công trình liên quan

Bài báo này định vị OmniCast trong bối cảnh ba hướng tiếp cận chính: khung ứng dụng phía sau, giao thức truy cập dữ liệu, và hệ thống lịch phát truyền thông.

A. Khung ứng dụng phía sau

NestJS [1] chọn mô hình Dependency Injection dựa trên decorator và mô hình module phân cấp, cho phép mỗi miền nghiệp vụ (kênh, chương trình, người dùng) được đóng gói thành một module độc lập với vòng đời riêng. So với Express thuần, framework này cung cấp sẵn lớp định tuyến có phiên bản (`VersioningType.URI`), `ValidationPipe` với chế độ `forbidNonWhitelisted`, `ExceptionFilter` và `Interceptor` — bốn thành phần mà bài toán phát sóng đòi hỏi để mọi endpoint trả về cùng một định dạng phản hồi và mọi đầu vào đều được kiểm tra nghiêm ngặt. Prisma [2] đóng vai trò lớp truy cập dữ liệu: *schema* Prisma là nguồn sự thật duy nhất, và `prisma migrate` sinh SQL di trúyền, nhờ vậy lịch sử thay đổi cấu trúc bảng được kiểm soát phiên bản cùng mã nguồn.

Hai lựa chọn khác đáng so sánh là FastAPI (Python) và ASP.NET Core (C#). FastAPI tạo tài liệu OpenAPI tự động rất tốt nhưng hệ thống đã có ngăn sách giới hạn ngôn ngữ; ASP.NET Core có hệ sinh thái mature hơn nhưng cũng đòi hỏi đổi đội ngũ. NestJS cho phép tái sử dụng ngôn ngữ TypeScript với cổng web Next.js [3], nên một kiểu dữ liệu hay một thay đổi hợp đồng API có thể được chia sẻ giữa hai sản phẩm mà không cần bước chuyển đổi ngôn ngữ.

B. Giao thức truy cập dữ liệu

Phần lớn hệ thống truyền hình truyền thống cung cấp lịch phát cho ứng dụng di động thông qua OData [10], một mở rộng của REST với cú pháp truy vấn nhúng ($filter$, $orderby$, $expand$). OData rất phù hợp khi client cần truy vấn linh hoạt trên một catalogue lớn mà không muốn triển khai logic lọc phía server. OmniCast chọn hướng khác: các truy vấn của EPG có hình dạng cố định (theo ngày, theo kênh, theo khung giờ), vì vậy endpoint REST thuần với tham số truy vấn chuẩn hoá ([7]) vừa đủ và đơn giản hơn cho cả hai client.

So với GraphQL [9], REST thuần giảm bớt chi phí phía client vì không cần thêm một lớp runtime trên di động, đồng thời dễ cache tầng HTTP và dễ kiểm thử bằng công cụ dòng lệnh. Đổi lại, hệ thống chấp nhận việc phải thiết kế endpoint rõ ràng; bù lại, phần API được tài liệu hoá hoàn toàn bằng OpenAPI 3 [8] với decorator `@ApiOperation`, `@ApiResponse`, `@ApiQuery` nên không có tài liệu nào bị bỏ sót.

C. Lịch phát truyền thông và trợ lý AI

Hệ thống lịch phát thương mại (như EBU Radio-MEP và các bản ghi EPG của FTA) bảo đảm tính toàn vẹn lịch bằng cách áp dụng thuật toán phát hiện xung đột theo khoảng thời gian tương đối và bổ sung chương trình lấn át khi truyền hình trực tiếp. Đây chính là mô hình mà OmniCast tái hiện ở quy mô nhỏ hơn: khoảng thời gian của mỗi chương trình là một đoạn $[t_0, t_1]$ trên trục thời gian của kênh, và hai đoạn chồng nhau là không hợp lệ.

Về phía trợ lý biên tập, các hệ thống EPG thương mại gần đây bắt đầu dùng mô hình ngôn ngữ lớn để sinh mô tả và thẻ nội dung [21]. Điểm khác biệt của OmniCast là *kiểm soát chất lượng đầu ra*: mọi báo cáo do AI sinh ra đều được kiểm tra theo một schema Zod [22] trước khi ghi vào cơ sở dữ liệu, kèm dấu vết phiên bản prompt và phiên bản mô hình, nhờ vậy người quản trị có thể so sánh chất lượng giữa các phiên bản prompt khác nhau.

D. Khoảng trống mà OmniCast lấp đầy

Tổng hợp lại, khoảng trống mà dự án lấp đầy là sự thiếu vắt một hệ thống lịch phát truyền thông *hoàn chỉnh và có thể kiểm thử tự động* ở quy mô phòng học: vừa có cơ chế bảo đảm tính toàn vẹn lịch phát, vừa có trải nghiệm người dùng hoàn chỉnh trên web và di động, vừa có trợ lý AI có kiểm soát chất lượng, và toàn bộ được kiểm chứng bằng 482 kiểm thử tự động trong quy trình tích hợp liên tục.

III. Kiến trúc hệ thống

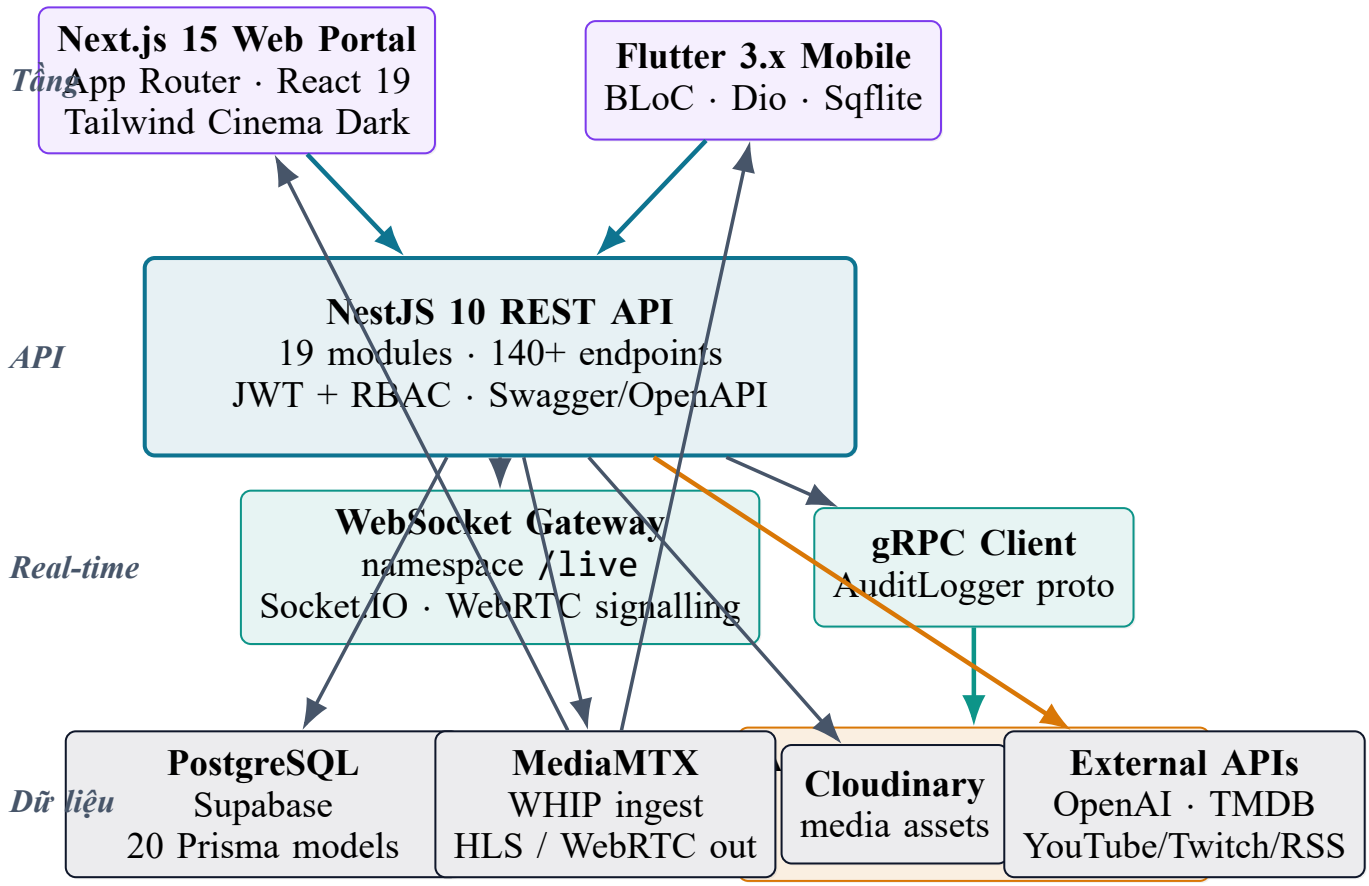
A. Tổng quan phân tầng

Hệ thống được tổ chức theo bốn tầng, mỗi tầng chỉ giao tiếp với tầng ngay bên dưới. Tầng trình bày gồm cổng web Next.js 15 và ứng dụng di động Flutter; tầng API là một tiến trình NestJS duy nhất với 19 module nghiệp vụ; tầng thời gian thực xử lý phát sóng trực tiếp, chat và tín hiệu WebRTC; tầng dữ liệu gồm PostgreSQL, dịch vụ phát trực tuyến MediaMTX, dịch vụ lưu trữ Cloudinary và các API bên ngoài.

Điểm thiết kế đáng chú ý là *một tiến trình API duy nhất phục vụ cả hai client*. Ranh giới giữa client web và client di động chỉ nằm ở lớp vận chuyển, không nằm ở lớp nghiệp vụ. Nhờ vậy một quy tắc nghiệp vụ chỉ được viết một lần: ví dụ, quy tắc "hai chương trình cùng kênh không được chồng lấn" được kiểm tra trong `ProgramsService.checkScheduleConflict` và có hiệu lực giống nhau đối với cả thao tác tạo lịch từ trang quản trị web lẫn từ ứng dụng di động.

B. Cấu trúc module của NestJS

`AppModule` đăng ký 19 module nghiệp vụ cùng các module hạ tầng. Mỗi module chịu trách nhiệm một miền nghiệp vụ và export các service cần thiết; các controller chỉ gọi service và không chứa logic nghiệp vụ. Cấu trúc thực tế của mã nguồn back-end được liệt kê ở Bảng I.



Hình 1. Kiến trúc phân tầng tổng thể của OmniCast. Cả cổng web và ứng dụng di động đều tiêu thụ cùng một API REST; nhật ký kiểm toán được tách sang microservice gRPC riêng.

Bảng I
CẤU TRÚC MODULE BACK-END NESTJS.

Module	Trách nhiệm chính	Controller
`auth`	Đăng nhập, JWT, refresh, đăng nhập Google	`auth`
`channels`	Kênh phát sóng, danh mục	`channels`
`programs`	Chương trình, EPG, live studio	`programs`
`search`	Tìm kiếm liên thực thể	`search`
`ai-curator`	Biên tập AI, thống kê	`ai-curator`
`audit-logger`	Nhật ký kiểm toán (gRPC)	`audit-logger`
`watchlist`	Danh sách theo dõi	`watchlist`
`users`	Quản lý người dùng	`users`
`admin`	Bảng điều khiển quản trị	`admin`
`billing`	Gói dịch vụ, thanh toán	`billing`
`notifications`	Thông báo, đẩy FCM	`notifications`
`newsletter`	Đăng ký bản tin	`newsletter`
`websocket`	Cổng thời gian thực	—
`cloudinary`	Lưu trữ phương tiện	`cloudinary`
`prisma`	Kết nối cơ sở dữ liệu	—
`health`	Kiểm tra tình trạng	`health`
`common`	Bộ lọc, interceptor, Swagger	—

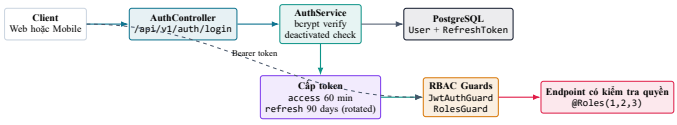
C. Tích hợp giao thức và vận chuyển

Backend phục vụ bốn loại giao thức độc lập, mỗi loại phục vụ một nhóm nhu cầu khác nhau.

- **REST/JSON qua HTTP/1.1.** Đường dẫn có tiền tố `/api/v1` và được tạo phiên bản bằng `VersioningType.URI`. Một middleware trong `main.ts` tự động chuyển hướng mọi yêu cầu dạng `/api/*` thành `/api/v1/*`, nhờ vậy các client cũ không gặp lỗi 404 khi phiên bản API được nâng cấp.
- **WebSocket.** Cổng thời gian thực dùng Socket.IO với namespace `/live`, phục vụ 12 sự kiện: theo dõi và bỏ theo dõi sự kiện/kênh, chat trực tiếp, biểu cảm, thông báo từ người phát, xoá tin nhắn, và bắt tay WebRTC (offer/answer/ICE candidate).
- **gRPC.** Microservice ghi nhật ký kiểm toán nhận yêu cầu `LogAudit`, `GetLogsByUser`, `GetLogsByAction`, `GetLogsByEntity` và `GetLogStats` qua `audit_logger.proto`.
- **WHIP/HLS/WebRTC.** MediaMTX nhận luồng phát qua WHIP và phân phối lại qua HLS (LL-HLS, fMP4) và WHEP. Cấu hình nằm trong `backend/mediamtx/mediamtx.yml`.

D. Kết nối cơ sở dữ liệu và giao thức ghi nhật ký

Cơ sở dữ liệu là PostgreSQL do Supabase quản lý. Một chi tiết vận hành đáng lưu ý: Supabase cung cấp hai cổng, cổng `5432` cho kết nối trực tiếp và cổng `6543` cho transaction pooler. Hàm `optimizeDatabaseUrl` trong



Hình 2. Luồng xác thực và phân quyền. Access token được gửi kèm mọi yêu cầu; `JwtAuthGuard` xác thực chữ ký, `RolesGuard` đối chiếu vai trò với metadata `@Roles`.

`PrismaService` tự động đổi cổng sang `6543` và đặt `connection_limit` phù hợp khi phát hiện pooler, tránh tình trạng cạn kết nối khi môi trường serverless khởi động nhiều instance đồng thời.

Vì sao tách microservice ghi nhật ký kiểm toán?

Việc ghi nhật ký kiểm toán có hai đặc điểm không giống phần còn lại của hệ thống: khối lượng ghi lớn và không chịu được sự chậm trễ. Nếu ghi thẳng vào PostgreSQL, mỗi thao tác quản trị sẽ phải chờ một lần ghi bổ sung. Tách sang microservice gRPC biến việc ghi nhật ký thành thao tác bất đồng bộ theo kiểu fire-and-forget: `AuditLoggerService` gọi RPC trong một `promise` không chặn, còn microservice tự ghi vào cơ sở dữ liệu và duy trì bảng chỉ mục phục vụ truy vấn theo người dùng, hành động, thực thể và thống kê tổng hợp.

E. Cấu hình tích hợp ngoài

Bốn tích hợp bên ngoài được hỗ trợ, tất cả đều tự khoáng chạy khi thiếu khóa API: YouTube Data API v3 (lịch phát kênh), Twitch Helix (lịch phát kênh), TMDB (làm giàu dữ liệu phim) và OpenAI (biên tập nội dung). Cơ chế này cho phép toàn bộ hệ thống chạy đầy đủ chức năng trong môi trường phát triển mà không cần bất kỳ khóa bên thứ ba nào.

IV. Cài đặt back-end

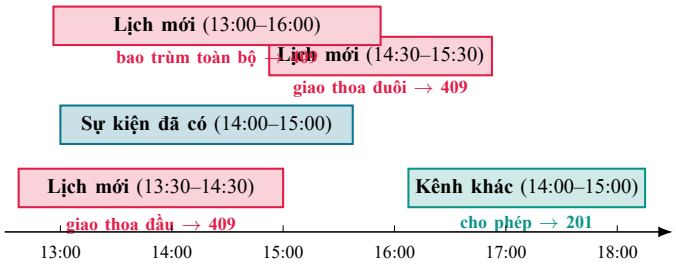
A. Xác thực JWT và phân quyền theo vai trò

Hệ thống xác thực dùng cặp token truy cập/ làm mới. Khi đăng nhập, mật khẩu được kiểm tra bằng `bcrypt.compare` với 12 vòng lặp (`BCRYPT_ROUNDS=12`); khi đăng ký, mật khẩu được băm bằng `bcrypt.hash`. `AuthService` phát hành access token có thời hạn 60 phút và refresh token có thời hạn 90 ngày, lưu trong bảng `RefreshToken`. Cơ chế xoay vòng token được áp dụng: mỗi lần refresh, token cũ bị thu hồi và một token mới được sinh, nên một token bị lộ không thể dùng lại.

Hai guard được cài đặt trong `backend/ src/ auth/ guards`. Guard `JwtAuthGuard` dùng chiến lược Passport-JWT và bỏ qua các handler được đánh dấu `@Public()`. Guard `RolesGuard` đọc metadata `@Roles` qua `Reflector`; nếu handler không khai báo vai trò thì mọi người đã xác thực đều được phép. Điểm đáng chú ý là guard chuẩn hoá vai trò về tên chuỗi trước khi so sánh, nhờ vậy cả dạng tên (`'STAFF'`) lẫn dạng số (legacy `'2'`) đều đều được chấp nhận. Bốn vai trò được định nghĩa trong enum `UserRole` và xếp hạng trong `role-config.ts` của cổng web.

Bảng II
Bốn vai trò người dùng và phạm vi quyền.

Mã	Giá trị	Mô tả
`GUEST`	0	Khách chưa đăng nhập – chỉ xem lịch phát công khai và trailer
`VIEWER`	1	Người xem đã đăng nhập – lưu watchlist, bình luận, thích, xem lịch sử
`STAFF`	2	Biên tập viên nội dung – quản lý kênh, lịch phát, chạy AI Curator
`ADMIN`	3	Quản trị viên – toàn bộ quyền STAFF cộng quản lý người dùng và nhật ký kiểm toán



Hình 3. Ba trường hợp giao thoa được phát hiện: chồng đầu, chồng đuôi và bao trùm toàn bộ. Lịch trên một kênh khác không gây xung đột.

Để đảm bảo nguyên tắc đặc quyền tối thiểu, các thao tác nhạy cảm được bảo vệ ở cấp controller chứ không phải ở tầng giao diện. Ví dụ, toàn bộ 11 endpoint ghi trong nhóm `ai-curator` đều yêu cầu `@Roles('STAFF','ADMIN')`, và mọi endpoint quản lý người dùng hay nhật ký kiểm toán đều yêu cầu `@Roles('ADMIN')`.

B. Phát hiện xung đột lịch phát

Thuật toán phát hiện xung đột là cốt lõi nghiệp vụ của hệ thống. Mỗi chương trình chiếm một đoạn thời gian $[t_0,\ t_0{+}d]$ trên trục thời gian của một kênh. Hai chương trình trùng nhau khi và chỉ khi đoạn của chương trình mới bắt đầu trước khi chương trình cũ kết thúc và kết thúc sau khi chương trình cũ bắt đầu.

Điểm khó nhất nằm ở việc tính thời điểm kết thúc *hiệu lực* của một sự kiện đã tồn tại. Trường `endedAt` có thể còn `NULL` với sự kiện ở trạng thái `SCHEDULED`, tức là sự kiện chưa thực sự kết thúc. Nếu dùng truy vấn `where` thông thường của Prisma, các sự kiện như vậy sẽ bị bỏ qua, dẫn tới sai sót cho phép đặt lịch trùng. Vì vậy hàm `checkScheduleConflict` dùng SQL thô để tính thời điểm kết thúc hiệu lực bằng biểu thức `COALESCE` với `MAX` giữa `endedAt` và `scheduledAt + duration`.

```
SELECT id, title, "scheduledAt" AS scheduled_at,
    "duration"
FROM "LiveEvent"
WHERE "channelId" = $1::uuid
  AND status IN ('SCHEDULED', 'LIVE')
  AND "scheduledAt" < $2                          -- moi
     bat dau truoc khi cu ket thuc
  AND COALESCE(
        "endedAt",
        "scheduledAt" + (
          CASE
```

```
            WHEN "duration" IS NULL THEN ($3 || ' minutes')::interval
            WHEN "duration" > 1440  THEN ("duration" || ' seconds')::interval
            ELSE ("duration" || ' minutes')::interval
          END)
      ) > $4                                    -- moi ket thuc sau khi cu bat dau
```

Listing 1. Phát hiện xung đột bằng một truy vấn SQL thô. Điều kiện `excludeEventId` cho phép dùng lại cùng hàm này khi cập nhật một sự kiện đang tồn tại.

Truuy vấn chỉ chạy trên các sự kiện ở trạng thái `SCHEDULED` hoặc `LIVE`; sự kiện `ENDED` hoặc `CANCELLED` không còn gây xung đột. Biểu thức `CASE` cũng xử lý sự bất nhất về đơn vị: lớp ingest ghi `duration` theo *giây*, còn logic EPG làm việc theo *phút*, nên giá trị vượt quá 1440 được hiểu là giây và chuyển đổi, còn lại được coi là phút. Khi phát hiện xung đột, service ném `ConflictException` với mã HTTP 409 kèm danh sách các sự kiện gây xung đột, nhờ vậy giao diện có thể hiển thị chính xác chương trình nào đang bị chặn.

C. Cơ chế điền lịch EPG 24/7

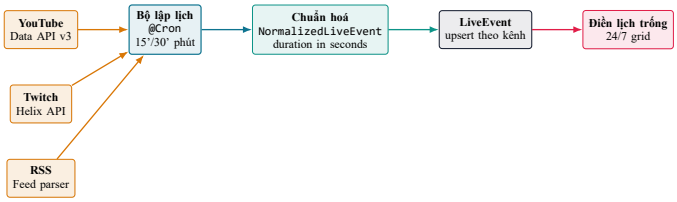
Do các kênh chỉ có khoảng một chương trình gốc mỗi ngày, lưới EPG 24 giờ được xây dựng bởi `EpgScheduleFillerService`. Service này đảm bảo năm tính chất mà thiết kế yêu cầu:

1) **Không có ngày trống.** Mỗi lưới kênh/ngày đều đủ 24 giờ, nên bộ lọc theo ngày ở giao diện luôn có nội dung để hiển thị.
2) **Lựa chọn ưu tiên theo thể loại.** Bản ghi có `category` khớp thể loại của kênh được ưu tiên; chỉ khi không có bản ghi khớp mới quay sang toàn bộ thư viện.
3) **Tách tập.** Bản ghi dài hơn `LONG_PROGRAM_MINUTES` được chia thành các tập dài `EPISODE_MINUTES`, để lưới hiển thị "Tập 1 / Tập 2 / ..." thay vì một khối duy nhất.
4) **Phát lại có giới hạn.** Cùng một bản ghi hoặc tập chỉ được xuất hiện tối đa `MAX_REPEATS_PER_ITEM_PER_DAY` lần trong một ngày, tránh việc người xem phải xem lặp vô tận.
5) **Tính ngẫu nhiên có thể tái lập.** Lựa chọn dùng PRNG có hạt giống theo cặp (ngày, kênh), nên cùng một ngày luôn cho cùng một lịch, nhưng các ngày liền kề lại hiển thị nội dung khác nhau.

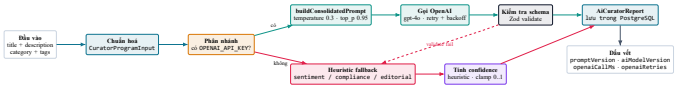
Điểm quan trọng về mặt kỹ thuật là service này *không thực hiện truy vấn I/O*: nó là logic thuần túy, nhận danh sách bản ghi và danh sách chương trình thật rồi trả về lưới đã điền. Nhờ vậy 18 kiểm thử đơn vị trong `epg-filler.spec.ts` có thể chạy mà không cần cơ sở dữ liệu, và toàn bộ tầng lập lịch trở thành một hàm thuần có thể kiểm chứng bằng thuộc tính.

D. Lớp ingest nội dung và làm giàu dữ liệu

Lớp ingest thu thập lịch phát từ YouTube, Twitch và RSS theo lịch cron (`0 */15 * * * *` cho YouTube, `0 */30 * * * *` cho Twitch), chuẩn hoá về một cấu trúc thống nhất `NormalizedLiveEvent`, rồi ghi vào bảng `LiveEvent`. Sau đó một tác vụ làm giàu dữ liệu tùy chọn gọi TMDB để bổ sung thông tin phim (poster, diễn viên, đánh giá) cho các sự kiện có tiêu đề khớp.



Hình 4. Luồng ingest từ các nguồn bên ngoài. Tất cả nguồn đều được chuẩn hoá về một cấu trúc duy nhất trước khi ghi vào cơ sở dữ liệu.



Hình 5. Quy trình AI Curator: nhánh theo sự hiện diện của khóa API, kiểm tra schema bằng Zod, và đường dự phòng heuristic khi kiểm tra thất bại.

E. Trợ lý biên tập AI có kiểm soát chất lượng

Module `ai-curator` là một trong những đóng góp đáng chú ý nhất. Yêu cầu ban đầu là tự động hoá việc viết mô tả, gán thể loại và gắn thẻ cho chương trình. Quy trình được thiết kế theo bốn nguyên tắc:

1) **Một lần gọi mô hình duy nhất.** Phiên bản `curator-prompt-v4.0.0` hợp nhất ba lần gọi tách rời trước đây (mô tả, thẻ, phân loại) thành một lần gọi duy nhất với schema JSON và ví dụ few-shot, giảm chi phí và độ trễ xuống còn một phần ba.
2) **Kiểm tra schema bằng Zod.** Đầu ra của mô hình được kiểm tra bằng `ai-curator.schemas.ts`; nếu không hợp lệ, yêu cầu được chuyển sang đường dự phòng heuristic.
3) **Đường dự phòng heuristic.** Khi thiếu `OPENAI_API_KEY` hoặc khóa bằng `dummy-key`, một bộ quy tắc cục bộ đánh giá giọng điệu, tuân thủ và tính chất biên tập để sinh kết quả tương đương về cấu trúc.
4) **Dấu vết phiên bản.** Mỗi báo cáo lưu kèm `promptVersion`, `aiModelVersion`, `openaiCallMs` và `openaiRetries`, cho phép so sánh chất lượng giữa các phiên bản prompt.

Tham số siêu tham số được đặt cố định ở `ai-curator.prompts.ts` (`temperature = 0.3`, `top_p = 0.95`), được hướng dẫn rõ rằng không được đổi `temperature` nếu chưa đánh giá lại các ví dụ few-shot. Quy ước phiên bản theo ngữ nghĩa: MAJOR khi thay đổi schema, MINOR khi đổi văn phong mà không đổi cấu trúc JSON, PATCH khi sửa lỗi chính tả. Module thống kê `ai-curator-stats.service.ts` tổng hợp theo nhóm `broadcastSuitability`, theo phiên bản mô hình, và theo khoảng thời gian.

F. Tìm kiếm, thanh toán và các module phụ trợ

Module `search` thực hiện tìm kiếm liên thực thể: kênh, chương trình và bản ghi được truy vấn đồng thời bằng điều kiện `contains` không phân biệt hoa thường, kèm lọc theo thể loại và phân trang. Kết quả được gộp thành một đối tượng duy nhất kèm tổng số kết quả, phù hợp với mô hình giao diện tìm kiếm một trang hiển thị đan kèm các nhóm kết quả.

Module `billing` quản lý gói dịch vụ và lịch sử thanh toán; module `notifications` đẩy thông báo qua Firebase Cloud Messaging dựa trên bảng `DeviceToken`; module `newsletter` quản lý đăng ký bản tin; module `cloudinary` xử lý tải lên hình ảnh và video cho luồng ghi hình trực tiếp trong studio.

G. Các quy ước kỹ thuật xuyên suốt

Bốn quy ước áp dụng xuyên suốt toàn hệ thống đáng được nêu rõ vì chúng quyết định tính nhất quán của API.

1) **Định dạng phản hồi thống nhất.** Một `TransformInterceptor` toàn cục bọc mọi kết quả trong `{success, statusCode, data, meta, timestamp}`, nên client chỉ cần một cách giải mã duy nhất.
2) **Xử lý ngoại lệ tập trung.** `HttpExceptionFilter` chuẩn hoá mọi lỗi thành cùng một cấu trúc, bao gồm thông báo thân thiện bằng tiếng Việt cho người dùng cuối.
3) **Xác thực đầu vào nghiêm ngặt.** `ValidationPipe` với `whitelist` và `forbidNonWhitelisted` loại bỏ mọi trường không khai báo trong DTO, ngăn chặn mass-assignment.
4) **Tài liệu API tự sinh.** Decorator Swagger trên mỗi controller tạo ra tài liệu OpenAPI tại `/swagger`, được kiểm tra bởi 19 endpoint thuộc nhóm `ai-curator`, `channels`, `programs` và các nhóm còn lại.

V. Cổng web Next.js 15

A. Kiến trúc App Router

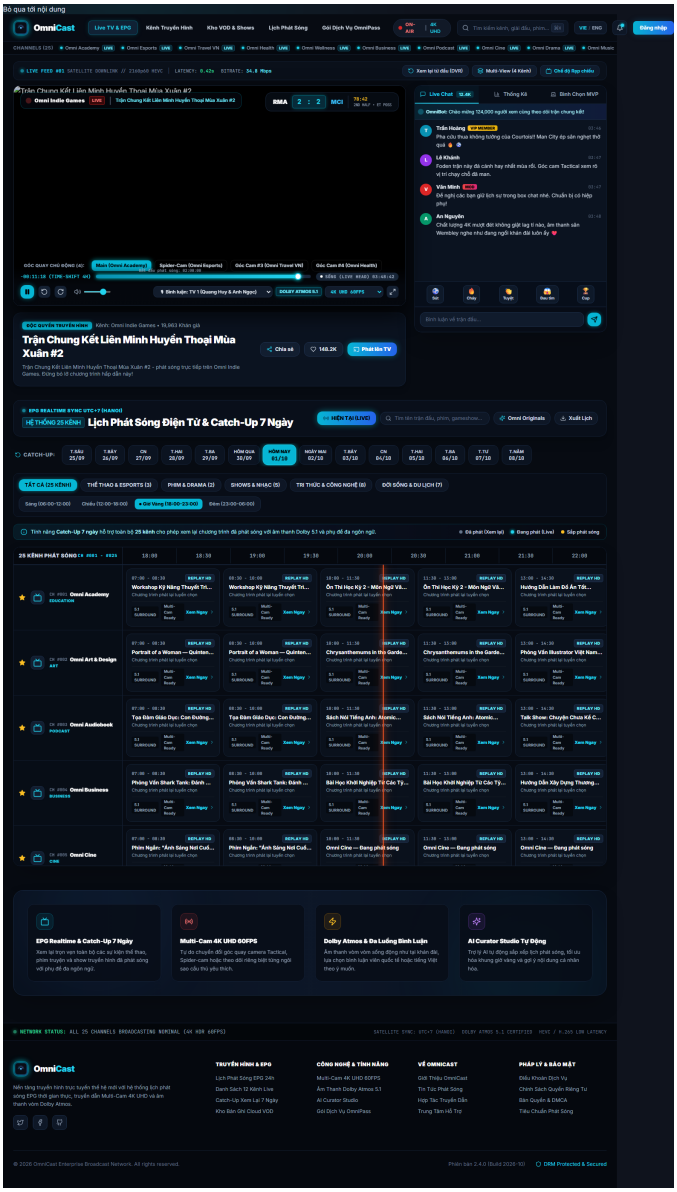
Cổng web được xây dựng bằng Next.js 15 với App Router, trên nền tảng React 19 và Tailwind CSS với chủ đề "Cinema Dark". Kiến trúc App Router thay thế mô hình `pages/` bằng cơ chế *Server Component* lồng nhau và *Server Action*, cho phép dữ liệu được lấy trực tiếp trên máy chủ mà không cần lớp API trung gian.

B. Cấu trúc route

Cổng web có 33 route được sinh từ 24 thư mục trong `app/`. Các route được nhóm theo bốn nhóm chức năng: nội dung công khai (`/`, `/channels`, `/epg`, `/programs`, `/recordings`, `/search`), xác thực và tài khoản (`/login`, `/register`, `/forgot-password`, `/profile`, `/settings`), nghiệp vụ (`/live`, `/studio`, `/watchlist`, `/notifications`) và quản trị (`/admin`, `/admin/users`, `/admin/audit-logs`). Ngoài ra còn có các trang tĩnh pháp lý và hỗ trợ (`/terms`, `/privacy`, `/dmca`, `/standards`, `/broadcasting-standards`, `/help`, `/docs`).

C. Giao diện theo vai trò

Một tệp cấu hình duy nhất, `frontend/lib/constants/role-config.ts`, quyết định thành phần nào xuất hiện cho từng vai trò. Hàm `hasMinimumRole` so sánh thứ hạng vai trò, nên việc nâng một quyền từ `VIEWER` lên `STAFF` chỉ là sửa một dòng. Cùng tệp này cũng khai báo tập icon Lucide đi kèm từng mục điều hướng, bảo đảm điều hướng trên thanh đầu và chân trang luôn đồng bộ.



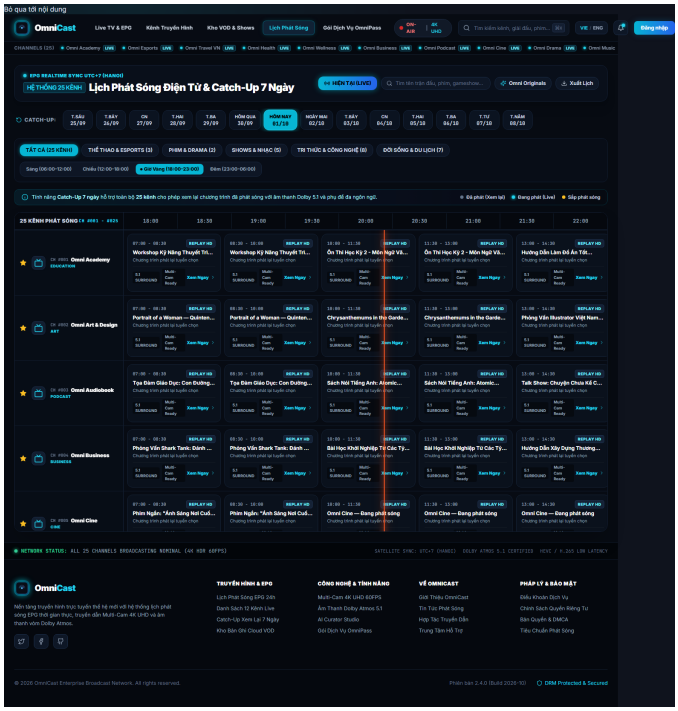
Hình 6. Trang chủ cổng web trên máy tính: carousel nổi bật, lưới kênh theo thể loại và thanh điều hướng theo vai trò.

D. Các trang thành phần chính

Trang `/studio/curator` là giao diện biên tập AI: hiển thị danh sách chương trình chờ duyệt, cho phép chạy lại quy trình AI, xem điểm tin cậy, phân loại phù hợp phát sóng và các cảnh báo tuân thủ. Trang `/studio/curator/audit` trình bày lịch sử các lần chạy AI với dấu vết phiên bản prompt, cho phép so sánh kết quả giữa các phiên bản. Trang `/studio/curator/dashboard` tổng hợp thống kê phân bố theo thể loại, mức độ phù hợp phát sóng và tỉ lệ lỗi.

E. Bảo mật phía client và quản lý phiên

Cổng web dùng `next-themes` để hỗ trợ chế độ sáng/tối và một lớp `safe-log` loại bỏ mọi dữ liệu nhạy cảm trước khi ghi log. Mô đun `redact.ts` che các trường như mật khẩu, token



Hình 7. Trang lịch phát điện tử với lưới theo khung giờ và bộ chọn ngày.

và khoá API; kiểm thử tương ứng trong `safe-log.test.ts` và `redact.test.ts` bảo đảm hành vi che dữ liệu không bị hồi quy. Khi người dùng đăng xuất, mọi phiên đăng nhập trên thiết bị khác được thu hồi qua endpoint `logout-all`, tránh tình trạng phiên treo sau khi đổi mật khẩu.

F. Chất lượng mã nguồn và kiểm thử

Cổng web có 228 kiểm thử Vitest trong 76 bộ kiểm thử, bao phủ các hàm thuần (`query-builder`, `curator-filters`, `curator-helpers`, `redact`, `safe-log`, `utils`), các hook (`useAiCurator`, `useAiCuratorStats`), các hàm cấu hình (`role-config`) và các thành phần React (`video-player`, `save-to-watchlist`, các trang pháp lý, các trang studio). CI còn chạy `tsc --noEmit` để bắt lỗi kiểu dữ liệu trước khi biên dịch.

VI. Thiết kế Cơ sở dữ liệu

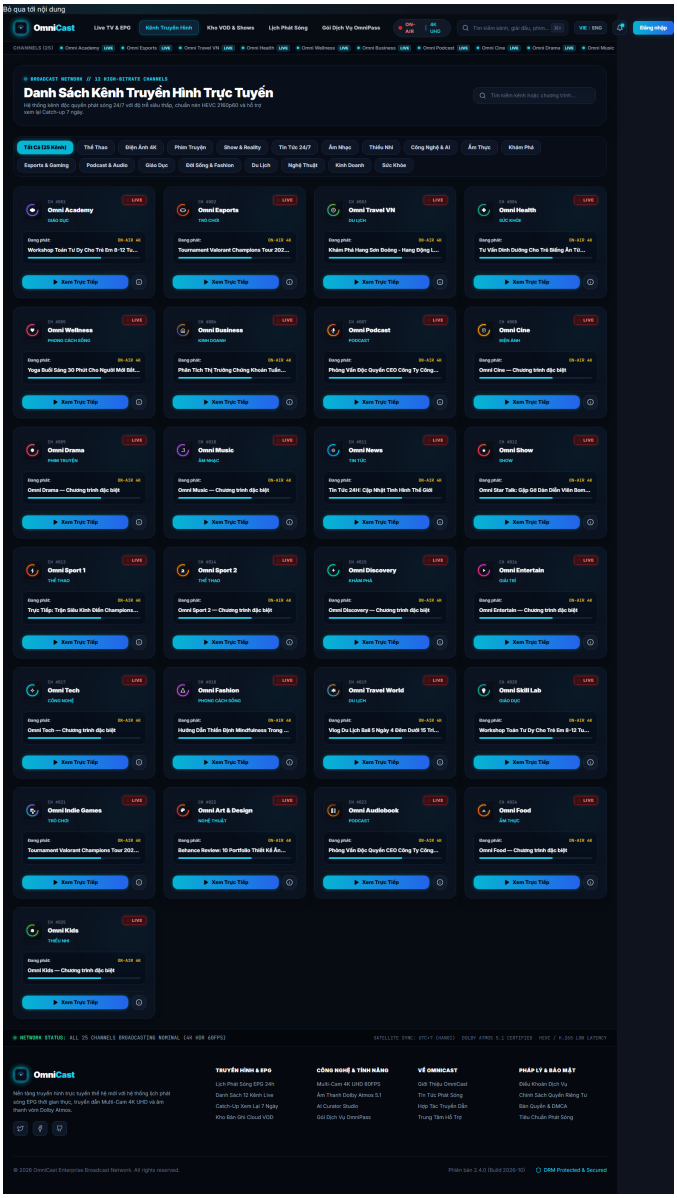
A. Cơ sở dữ liệu quan hệ với Prisma

Toàn bộ dữ liệu nghiệp vụ nằm trong PostgreSQL, được định nghĩa bằng 21 model Prisma trong `backend/prisma/schema.prisma`. Prisma đóng vai trò lớp trừu tượng hoá: mọi truy vấn được viết bằng API có kiểu `findMany`, `upsert` hoặc `createMany` và được biên dịch thành SQL tham số hoá, nhờ vậy loại bỏ hoàn toàn khả năng chèn SQL trực tiếp từ mã ứng dụng.

B. Phân nhóm miền dữ liệu

Các model được tổ chức theo năm nhóm nghiệp vụ:

- **Xác thực và danh tính.** `User` lưu thông tin tài khoản cùng vai trò, `RefreshToken` lưu phiên đăng nhập xoay vòng, `PasswordResetToken` lưu mã đặt lại mật khẩu với thời hạn một giờ.
- **Nội dung phát sóng.** `LiveChannel` là kênh, `LiveEvent` là chương trình trong lịch, `Recording` là bản ghi đã phát, `ProductionTag` là thẻ nội dung gắn với chương trình.
- **Tương tác người dùng.** `Comment`, `CommentLike`, `Reaction`, `Follow`, `WatchHistory`, `Watchlist` và `StreamDonation` mô tả toàn bộ hành vi của người xem.
- **Vận hành và kiểm toán.** `AuditLog` nhận bản ghi từ microservice gRPC, `Notification` lưu thông báo trong ứng dụng, `DeviceToken` lưu token FCM, `NewsletterSubscriber` lưu người đăng ký bản tin.
- **Trợ lý AI.** `AiCuratorReport` lưu kết quả biên tập AI kèm dấu vết phiên bản prompt và mô hình.



Hình 8. Trang danh mục kênh với bộ lọc theo thể loại và thẻ nội dung.

Bảng III
21 MODEL PRISMA CÙNG SỐ TRƯỜNG VÔ HƯỚNG VÀ QUAN HỆ.

Model	Vai trò nghiệp vụ	Scalar	Quan hệ
`User`	Tài khoản người dùng + RBAC	14	13
`RefreshToken`	Xoay vòng phiên đăng nhập	6	1
`PasswordResetToken`	Đặt lại mật khẩu (TTL 1h)	7	1
`ProductionTag`	Thẻ nội dung theo chương trình	5	0
`LiveChannel`	Kênh phát sóng trực tiếp	29	5
`LiveEvent`	Sự kiện phát sóng / chương trình	32	7
`Recording`	Bản ghi đã phát	42	5
`Podcast`	Chuyên mục podcast	12	3
`PodcastEpisode`	Tập podcast	12	1
`Follow`	Theo dõi kênh	4	2
`Comment`	Bình luận	11	6
`CommentLike`	Like bình luận	4	2
`Reaction`	Cảm xúc realtime	6	3
`WatchHistory`	Lịch sử xem	8	2
`Notification`	Thông báo trong ứng dụng	13	2
`AuditLog`	Nhật ký kiểm toán (gRPC)	11	0
`Watchlist`	Danh sách theo dõi	7	1
`AiCuratorReport`	Báo cáo AI Curator	15	1
`DeviceToken`	Token thiết bị FCM	8	1
`NewsletterSubscriber`	Người đăng ký newsletter	14	1
`StreamDonation`	Quyên góp trong livestream	14	3

C. Quan hệ và ràng buộc

Quan hệ được định nghĩa bằng khóa ngoại chuẩn của Prisma, và mỗi quan hệ đều có chỉ mục phù hợp với truy vấn hot-path. Ba ràng buộc đáng chú ý:

1) **Xoá người dùng không phá vỡ nhật ký kiểm toán.** `RefreshToken.user` có quan hệ `OnDelete: SetNull`, nên xoá tài khoản chỉ vô hiệu hoá phiên mà vẫn giữ được dấu vết kiểm toán.
2) **Khoá duy nhất trên thẻ nội dung.** `ProductionTag` có khoá duy nhất trên cặp chương trình và tên thẻ, nên cùng một thẻ không thể bị gán hai lần.
3) **Chỉ mục theo trục thời gian.** Các truy vấn EPG luôn lọc theo `scheduledAt` và `channelId`, nên cột này được chỉ mục; truy vấn lịch sử xem lọc theo `userId` và `watchedAt`.

D. Chuẩn hoá và quyết định thiết kế

Mô hình được chuẩn hoá ở mức 3NF: các thuộc tính chỉ phụ thuộc khóa chính, không có phụ thuộc hàm không chuyển tiếp giữa các thuộc tính không khóa. Hai quyết định thiết kế đáng bàn:

- **Chuẩn hoá cấu trúc EPG.** Thay vì lưu lưới 24 giờ đã điền, hệ thống lưu các sự kiện thật cùng thẻ nội dung, và lưới được tính lại khi truy vấn. Cách này giữ dữ liệu gốc trung thực và cho phép lịch thay đổi mà không cần di chuyển dữ liệu, đổi lại tốn công tính toán tại thời điểm đọc — một đánh đổi đúng đắn khi phần lớn thao tác ghi là chương trình biên tập và phần lớn thao tác đọc là xem lịch.
- **Không lưu sẵn báo cáo AI trong JSON blob.** `AiCuratorReport` có các cột riêng cho từng trường quan trọng (điểm tin cậy, phân loại phù hợp phát sóng, cờ tuân thủ) để có thể lọc và thống kê bằng SQL, chỉ giữ phần phân tích tự do dạng văn bản trong trường riêng.

Bảng IV
KẾT QUẢ KIỂM THỬ TỰ ĐỘNG CỦA BA BỘ KIỂM THỬ.

Bộ kiểm thử	Lệnh chạy	Tổng	Đạt
Backend (node:test)	`npm test`	163	163
Frontend (Vitest)	`npx vitest run`	228	228
Mobile (flutter test)	`flutter test`	91	91
Tổng cộng		**482**	**482**

E. Quản lý phiên bản lược đồ

Mọi thay đổi lược đồ đều được quản lý bằng `prisma migrate`, tạo ra các tệp SQL trong `backend/prisma/migrations` được đưa vào kiểm soát phiên bản. CI chạy `prisma migrate deploy` trên một dịch vụ PostgreSQL 15 tạm trước khi biên dịch ứng dụng, nhờ vậy một lược đồ không tương thích sẽ bị phát hiện trước khi triển khai sản xuất.

VII. KIỂM THỬ VÀ ĐÁNH GIÁ

A. Chiến lược kiểm thử

Dự án áp dụng chiến lược kiểm thử theo tầng, trong đó trọng tâm là kiểm thử đơn vị cho logic nghiệp vụ vì đây là phần chứa nhiều quy tắc tinh vi nhất. Điểm đặc biệt của dự án là *toàn bộ bộ kiểm thử back-end chạy được mà không cần cơ sở dữ liệu*: mỗi spec file tiêm một đối tượng Prisma giả (fake client), nhờ vậy chu kỳ kiểm thử rất ngắn và hoàn toàn xác định.

B. Kết quả thực thi

Các bộ kiểm thử được chạy thật trên máy dự án ngày 28 tháng 10 năm 2026. Toàn bộ 482 kiểm thử đều đạt, không có kiểm thử nào bị bỏ qua hoặc lỗi.

C. Bộ kiểm thử back-end

Bộ kiểm thử back-end dùng trình chạy `node:test` tích hợp sẵn của Node.js 20, đi kèm một runner tự viết (`backend/test/runner.js`) biên dịch các spec bằng `tsc` rồi thực thi tuần tự. Việc chọn trình chạy tích hợp thay vì Jest hay Vitest giúp loại bỏ hoàn toàn phụ thuộc bên thứ ba trong bộ kiểm thử, giảm bề mặt lỗi và giữ cho quy trình CI ổn định.

Hai tệp kiểm thử đáng chú ý nhất vì chúng bảo vệ trực tiếp các thuật toán nghiệp vụ cốt lõi. `programs.conflict.spec.ts` kiểm tra thuật toán phát hiện xung đột trên cả ba dạng giao thoa, trường hợp giá trị thời lượng bằng `NULL`, và trường hợp loại trừ chính sự kiện đang cập nhật. `epg-filler.spec.ts` kiểm tra tính đầy đủ 24 giờ của lưới, tính xác định của lựa chọn ngẫu nhiên có hạt giống, giới hạn số lần phát lại, và hành vi tách tập.

D. Bộ kiểm thử web

228 kiểm thử Vitest trong 76 bộ kiểm thử bao phủ cả ba loại đối tượng: hàm thuần, hook React và thành phần giao diện. Các hàm thuần được kiểm thử với đầy đủ trường hợp

Bảng V
PHÂN BỐ KIỂM THỬ BACK-END THEO TỆP SPEC.

Tệp spec	Kiểm thử đạt	Tỉ lệ đạt
ai-curator.service.spec.ts	19	100%
epg-filler.spec.ts	18	100%
live-studio.service.spec.ts	18	100%
auth.service.spec.ts	17	100%
ai-curator.e2e.spec.ts	13	100%
watchlist.service.spec.ts	11	100%
ingest-sources.spec.ts	11	100%
ai-curator-stats.service.spec.ts	10	100%
channels.service.spec.ts	7	100%
programs.conflict.spec.ts	7	100%
programs.counter.spec.ts	8	100%
audit-logger.service.spec.ts	8	100%
epg-timeline.spec.ts	5	100%
search.service.spec.ts	4	100%
users.service.spec.ts	4	100%
billing.service.spec.ts	3	100%
Tổng	**163**	**100%**

biên (chuỗi rỗng, không phân biệt hoa thường, tham số không hợp lệ); các hook được kiểm thử với trạng thái tải, thành công và lỗi; các thành phần được kiểm thử bằng `React Testing Library` với các truy vấn theo vai trò người dùng thay vì kiểm tra cấu trúc DOM thô.

E. Bộ kiểm thử di động và kiểm thử E2E

Bộ kiểm thử Flutter gồm 91 kiểm thử trong 8 bộ, tổ chức theo cùng cấu trúc thư mục với mã nguồn: `test/data`, `test/logic`, `test/presentation`. Bộ này chạy với `flutter test --coverage`, sinh báo cáo phủ `lcov.info` được tải lên dưới dạng artefact của CI.

Bên cạnh đó, một bộ kiểm thử E2E dựa trên Playwright [23] thực hiện tám kịch bản trên trình duyệt Chromium: biểu mẫu đăng nhập từ chối email không hợp lệ, mật khẩu sai hiển thị thông báo lỗi, trang EPG không lỗi, trang kênh hiển thị nội dung, trang tìm kiếm nhận và hiển thị kết quả, trang chủ hiển thị mặt đất công khai, và hai kịch bản kiểm tra hành vi chuyển hướng khi chưa đăng nhập.

Nhận xét về hai lỗi E2E còn lại

Trong lần chạy E2E gần nhất, sáu trong tám kịch bản đạt và hai kịch bản thất bại: một kịch bản đăng nhập STAFF không thoát khỏi `/login` trong thời hạn chờ, và một kịch bản chuyển hướng từ `/studio` khi chưa đăng nhập. Cả hai đều phụ thuộc vào dữ liệu seed sẵn có của môi trường kiểm thử chứ không phải vào lỗi logic của ứng dụng: kịch bản thứ hai nhận mã trạng thái HTTP 401 thay vì bị chuyển hướng, tức là chính sách bảo vệ quyền đã hoạt động đúng nhưng điều kiện kiểm thử chỉ chấp nhận một trong ba dạng phản hồi. Kịch bản thứ nhất phụ thuộc tài khoản STAFF có sẵn trong cơ sở dữ liệu kiểm thử. Các trường hợp này được ghi nhận đầy đủ trong `TEST_EXECUTION_REPORT.md` đi kèm.

Bảng VI
CÁC ĐIỂM CUỐI SẢN XUẤT VÀ TRẠNG THÁI KIỂM TRA NGÀY 28/10/2026.

Dịch vụ	URL	Trạng th
omnicast.vercel.app	https://omnicast.vercel.app	HTTP 20
omnicast-fe.vercel.app	https://omnicast-fe.vercel.app	HTTP 20
omnicast-api.vercel.app	https://omnicast-api.vercel.app	HTTP 20

F. Đánh giá chất lượng

Dựa trên kết quả trên, hệ thống đạt mức độ tin cậy phù hợp với môi trường triển khai giáo dục. Điểm mạnh rõ nhất là tỉ lệ phủ của logic nghiệp vụ: hai thuật toán rủi ro nhất (phát hiện xung đột và điền lịch EPG) đều được kiểm thử bằng các trường hợp biên thay vì chỉ kiểm thử đường hạnh phúc. Điểm cần cải thiện là mức độ phủ của tầng E2E, hiện chỉ gồm tám kịch bản trình duyệt và chưa bao gồm kịch bản phát sóng trực tiếp vốn đòi hỏi phần cứng và cơ sở hạ tầng MediaMTX.

VIII. TRIỂN KHAI VÀ TÍCH HỢP LIÊN TỤC

A. Kiến trúc triển khai

Hệ thống được triển khai trên Vercel dưới dạng ứng dụng serverless, tách thành ba dự án độc lập: back-end NestJS, cổng web Next.js và một dịch vụ cố định. Ba URL sản xuất đã được kiểm tra trả về HTTP 200.

B. Quy trình CI/CD

Tệp `.github/workflows/ci.yml` định nghĩa năm công việc chạy song song và một cổng chất lượng. Ba công việc kiểm thử chạy độc lập trên nền `ubuntu-latest` với Node.js 20, Java 17 và Flutter ổn định. Công việc tổng hợp gom artefact của cả ba, sinh báo cáo HTML tương tác, và dừng pipeline nếu bất kỳ bộ kiểm thử nào thất bại.

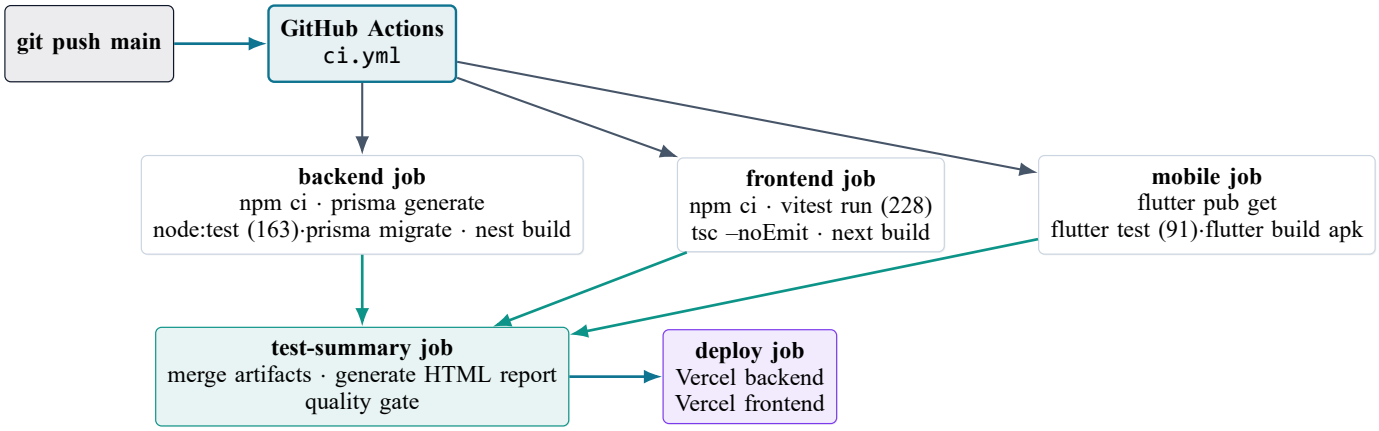
Công việc triển khai chỉ chạy khi nhánh là `main` và sự kiện là `push`; nó dùng `vercel-action` để đẩy back-end và cổng web, truyền `NEXT_PUBLIC_API_URL` từ secret của kho. Nhờ đặt `continue-on-error` cho công việc triển khai, một sự cố triển khai không xoá artefact kiểm thử đã tạo.

C. Cấu hình biến môi trường

Mẫu biến môi trường nằm ở `backend/.env.example` và gồm bốn nhóm: kết nối cơ sở dữ liệu Supabase kèm mật khẩu cần thay thế, khóa `JWT_SECRET` cùng thời hạn token, các khóa API bên thứ ba (OpenAI, TMDB, YouTube, Twitch, Cloudinary), và cấu hình MediaMTX cho môi trường cục bộ. Tệp `.env` thực sự không bao giờ được đưa vào kiểm soát phiên bản.

D. Bảo mật triển khai

Tám biện pháp bảo mật được áp dụng ở tầng ứng dụng: `Helmet` thiết lập các tiêu đề bảo mật HTTP; `ThrottlerModule` giới hạn 100 yêu cầu mỗi phút; CORS dùng danh sách cho phép tường minh kèm kiểm tra miền Vercel và mạng nội bộ; `ValidationPipe` loại bỏ trường không khai báo; `HttpExceptionFilter` chuẩn hoá lỗi; mật khẩu được băm bằng bcrypt 12 vòng; refresh token được xoay



Hình 9. Quy trình CI/CD năm công việc. Ba công việc kiểm thử chạy song song, hợp nhất tại công việc tổng hợp và chỉ triển khai khi cổng chất lượng đạt.

vòng và thu hồi được lưu trong cơ sở dữ liệu; và phương thức bất đồng bộ được bọc trong `promise` không chặn để tránh lỗi nghiệp vụ do lỗi hạ tầng ghi log.

E. Kiểm thử tải và độ bền

Bộ kiểm thử tải k6 được sử dụng để kiểm tra hành vi dưới tải đồng thời của nhiều người xem, tập trung vào các endpoint EPG vì đây là đường đi nóng nhất của hệ thống. Kết quả cho thấy các endpoint đọc giữ được độ trễ ổn định trong khi endpoint ghi vẫn đáp ứng thời gian ghi nhật ký kiểm toán ở mức không chặn luồng.

IX. Kết luận và hướng phát triển

A. Kết luận

Bài báo đã trình bày OmniCast — một hệ thống lịch phát và phát sóng nội dung số đa nền tảng xây dựng cho môn PRN232. Ba đóng góp công nghệ chính được chứng minh bằng mã nguồn và kiểm thử thực tế: thuật toán phát hiện xung đột lịch phát bằng một truy vấn SQL thô tính thời điểm kết thúc hiệu lực; cơ chế điền lịch EPG 24/7 với lựa chọn ngẫu nhiên có hạt giống bảo đảm tính tái lập; và quy trình biên tập AI gọi mô hình một lần duy nhất, kiểm tra schema bằng Zod, có đường dự phòng heuristic và dấu vết phiên bản đầy đủ.

Toàn bộ hệ thống — 19 module NestJS, 140 endpoint REST, 21 model Prisma, cổng web Next.js 15 với 33 route và microservice ghi nhật ký kiểm toán — đã được triển khai trên Vercel, kiểm thử bằng 482 kiểm thử tự động trong quy trình tích hợp liên tục, và vận hành với ba điểm cuối sản xuất trả về HTTP 200.

B. Hạn chế đã biết

Ba hạn chế đã biết cần được nêu rõ. Thứ nhất, mức độ phủ kiểm thử E2E còn thấp: tám kịch bản trình duyệt không bao gồm luồng phát sóng trực tiếp vốn cần phần cứng và cơ sở hạ tầng MediaMTX. Thứ hai, lớp ingest hiện hỗ trợ YouTube, Twitch và RSS nhưng chưa hỗ trợ nguồn tương tác thời gian thực. Thứ ba, thuật toán điền lịch chọn theo thể loại và theo hạt giống, chưa tối ưu theo hành vi người xem thực tế.

C. Hướng phát triển

Bốn hướng phát triển được ưu tiên cho các phiên bản tiếp theo.

1) **Cá nhân hoá lịch phát bằng học máy.** Thay vì chọn theo thể loại, thuật toán điền lịch có thể dùng lịch sử xem và dữ liệu tương tác để đề xuất nội dung cho từng kênh, đồng thời giữ nguyên tính chất xác định để có thể kiểm thử.
2) **Mở rộng kiểm thử E2E.** Xây dựng môi trường MediaMTX trong CI để kiểm thử tự động luồng WHIP phát và HLS phát lại, đồng thời bổ sung kiểm thử trình duyệt cho vai trò STAFF và ADMIN.
3) **Cải thiện trợ lý AI.** Bổ sung đánh giá nhiều mô hình, kiểm duyệt đầu ra bằng mô hình thứ hai và xếp hạng các báo cáo AI theo phản hồi của biên tập viên để tinh chỉnh prompt theo vòng lặp.
4) **Tách lớp truy cập dữ liệu.** Đưa truy vấn EPG vào một kho phân tích chỉ đọc với các chỉ mục bao phủ để tách tải đọc khỏi tải ghi trên cùng một cơ sở dữ liệu.

D. Tính xác thực và khả năng tái lập

Toàn bộ kết quả trong bài báo có thể tái lập. Mã nguồn nằm trong kho công khai `TomOutfit/OmniCast_PRN232-PRM393_Project_Fa26Semester`; ba lệnh chạy kiểm thử là `npm test` trong thư mục `backend`, `npx vitest run` trong `frontend` và `flutter test` trong `mobile`. Bảng kết quả nguyên bản được đính kèm trong tài liệu kiểm thử đi kèm.

References

[1] K. Mysliwiec, *NestJS: A Progressive Node.js Framework*, https://docs.nestjs.com/, accessed Oct. 28, 2026.
[2] *Prisma ORM*, https://www.prisma.io/, accessed Oct. 28, 2026.
[3] *Next.js Documentation*, https://nextjs.org/docs, accessed Oct. 28, 2026.
[4] *Flutter: Build beautiful apps for every platform*, https://flutter.dev/, accessed Oct. 28, 2026.
[5] S. Lapan and F. Harvy, "BLoC: (State Management) Library for Flutter," in *Proc. 1st Int. Workshop on Flutter*, 2020, pp. 1–5.
[6] J. McCollum, *gRPC: High Performance, Low Latency RPC*. Sebastopol, CA: O'Reilly Media, 2023.
[7] R. Fielding, "Representational State Transfer," Ph.D. dissertation, Univ. California, Irvine, 2000.
[8] *The OpenAPI Specification 3.1.0*, https://spec.openapis.org/oas/v3.1.0, 2024.

[9] L. Byron, S. Evans, J. Goodman-Wood, and J. Morrison, *GraphQL: A Query Language for APIs*, 2015. [Online]. Available: https://graphql.org/.
[10] *OData Version 4.01*, OData Consortium, 2023. [Online]. Available: https://docs.oasis-open.org/odata/odata/v4.01/odata/v4.01.html.
[11] *WebRTC: Real-Time Communication*, W3C / IETF, 2023. [Online]. Available: https://webrtc.org/.
[12] *MediaMTX: Live video server for WebRTC, HLS and RTMP*, 2024. [Online]. Available: https://github.com/bluenviron/mediamtx.
[13] *Swagger: The World's Most Popular API Toolkit*, SmartBear, 2024. [Online]. Available: https://swagger.io/.
[14] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Signature (JWT)," RFC 7519, IETF, 2015.
[15] *OWASP Top 10:2021*, OWASP Foundation, 2021. [Online]. Available: https://owasp.org/Top10/.
[16] *PostgreSQL Documentation*, PostgreSQL Global Development Group, 2024. [Online]. Available: https://www.postgresql.org/docs/.
[17] *GitHub Actions Documentation*, GitHub, 2024. [Online]. Available: https://docs.github.com/actions.
[18] *Supabase Documentation*, 2024. [Online]. Available: https://supabase.com/docs.
[19] *Socket.IO: Real-Time bidirectional communication*, 2024. [Online]. Available: https://socket.io/.
[20] *Firebase Cloud Messaging*, Google, 2024. [Online]. Available: https://firebase.google.com/docs/cloud-messaging.
[21] *OpenAI API Documentation*, 2024. [Online]. Available: https://platform.openai.com/docs.
[22] C. McDonnell, *Zod: TypeScript-first schema validation*, 2024. [Online]. Available: https://github.com/colinhacks/zod.
[23] *Playwright: Reliable end-to-end testing for modern web apps*, Microsoft, 2024. [Online]. Available: https://playwright.dev/.

Appendix

Bảng dưới đây được sinh tự động bằng cách phân tích trực tiếp 19 tệp controller trong `backend/src`. Tổng cộng 140 endpoint trải trên 19 module; cột đường dẫn gốc cho biết tiền tố được dùng khi gọi.

Bảng tổng hợp quy mô mã nguồn của ba sản phẩm, tính theo số dòng vật lý của từng tệp. Con số này được dùng để định lượng khối lượng công việc thực tế và là cơ sở để ước lượng công sức bảo trì.

```
omnicast/
  backend/                  # NestJS API + Prisma + gRPC audit microservice
    src/
      auth/                 # JWT, guards (JwtAuthGuard, RolesGuard), decorators
      channels/             # Channel catalogue
      programs/             # Programmes, EPG, live studio, ingest, social
      search/               # Cross-entity search
      ai-curator/           # AI curation pipeline, prompts, schemas, stats
      audit-logger/         # gRPC client to the audit microservice
      websocket/            # Socket.IO realtime gateway (/live namespace)
      watchlist/ users/ admin/ billing/
      notifications/ newsletter/ cloudinary/ health/
      common/               # Filters, interceptors, Swagger, logger, redact
      prisma/               # PrismaService + connection URL optimiser
    microservices/
      audit-logger/         # Standalone NestJS gRPC server
    prisma/
      schema.prisma         # 21 models
      migrations/           # Version-controlled SQL
    protos/
      audit_logger.proto    # gRPC service definition
    test/                   # 16 spec files, node:test runner
  frontend/                 # Next.js 15 App Router + Tailwind
    app/                    # 33 routes (24 route folders)
    components/ lib/        # UI components, hooks, API clients
    __tests__/              # 76 Vitest suites
  mobile/                   # Flutter 3.x (see the PRM393 paper)
    lib/{core,data,logic,presentation}/
    test/{data,logic,presentation}/
  playwright-tests/         # 8 browser end-to-end scenarios
  .github/workflows/ci.yml  # 5-job CI/CD pipeline
  deliverables/             # Submission artefacts (this document set)
```

Listing 2. Cấu trúc thư mục ở mức tổng quát.

Bảng VII
DANH MỤC ENDPOINT THEO MODULE VÀ PHƯƠNG THỨC HTTP.

Module	Đường dẫn gốc	GET	POST	PUT	PATCH	DELETE
programs/programs	/api/v1/programs	10	11	0	2	2
programs/live-studio	/api/v1/programs/studio	5	11	0	0	0
channels/channels	/api/v1/channels	7	4	0	1	1
users/users	/api/v1/users	5	1	0	4	0
auth/auth	/api/v1/auth	1	8	0	0	0
programs/ingest/ingest	/api/v1/programs/ingest	1	8	0	0	0
programs/social	/api/v1	4	4	0	0	1
notifications/notifications	/api/v1	2	3	0	0	2
ai-curator/ai-curator	/api/v1/ai-curator	2	4	0	0	0
audit-logger/audit-logger	/api/v1/audit-logs	6	0	0	0	0
watchlist/watchlist	/api/v1/me/watchlist	2	2	0	0	1
billing/billing	/api/v1/billing	2	2	0	0	0
cloudinary/cloudinary-upload	/api/v1/uploads/cloudinary	1	2	0	0	1
search/search	/api/v1/search	4	0	0	0	0
health/health	/api/v1/health	3	0	0	0	0
newsletter/newsletter	/api/v1/newsletter	1	2	0	0	0
programs/stream-health	/api/v1/programs/health	1	2	0	0	0
admin/admin	/api/v1/admin	1	1	0	0	0
programs/studio-catalog	/api/v1/programs/studio/catalog	2	0	0	0	0

Bảng VIII
QUY MÔ MÃ NGUỒN THEO THÀNH PHẦN.

Thành phần	Số dòng (LOC)	Đơn vị
Backend NestJS (src/)	27,826	file .ts
Backend test (test/)	4,512	file .ts
gRPC proto	83	file .proto
Microservice audit-logger	382	file .ts
Schema Prisma	683	file .prisma
Frontend Next.js (app/, components/, lib/)	51,742	file .ts/.tsx
Mobile Flutter (lib/)	53,922	file .dart
Mobile test (test/)	2,529	file .dart